University of Calgary

Department of Electrical and Software Engineering

ENSF 694 – Data Structures and Algorithms

Term Project

Campus Navigation and Event Management System

Course	ENSF 694 – Data Structures and Algorithms
Term	Summer 2026
Programming Language	C++
Total Weight	30% of Final Grade

1. Problem Description

Modern university campuses are complex environments where students, faculty, and visitors must frequently navigate between buildings, book rooms, and keep track of scheduled events. Managing this efficiently requires intelligent software capable of handling spatial data, scheduling constraints, and rapid information lookup — all simultaneously.

In this project, you will design and implement a Campus Navigation and Event Management System for a fictional (or simplified real) university campus. Your system must model the campus as a network of buildings and pathways, compute optimal routes between locations, manage room bookings and events, and provide fast access to campus information.

The challenge lies not only in implementing the required features, but in making deliberate and well-justified choices about the underlying data organization and algorithms that power each feature. You are expected to think critically about efficiency: what is the time and space complexity of your solution, and why are your choices appropriate given the expected usage patterns?

By the end of this project, you will have built a cohesive, working application that integrates multiple core concepts from the course into a single, meaningful system.

1.1 Representing the Campus Environment

Your system must model the physical campus using a hierarchy of Python classes. The following structure is provided as a starting point — you are free to extend it, but the core relationships must be preserved. (note that code provided is in Python and should be converted to C++)

```
class Room:
    def __init__(self, room_id: str, capacity: int, room_type: str):
        self.room_id   = room_id      # e.g. "ICT-121"
        self.capacity  = capacity     # max occupancy
        self.room_type = room_type    # "lecture", "lab", "office"
        self.bookings  = []           # list of Booking objects
class Building:
    def __init__(self, building_id: str, name: str, location: tuple):
        self.building_id = building_id   # e.g. "ICT"
        self.name        = name          # "Information and Comm. Tech."
        self.location    = location      # (lat, lon) or grid coords
        self.rooms       = {}            # room_id -> Room
```

A Campus object then aggregates all buildings and owns the pathway network that connects them:

```
class Campus:
    def __init__(self):
        self.buildings = {}      # building_id -> Building
        self.pathways  = ...     # your chosen network representation
```

The diagram below illustrates a sample 8-building campus map with weighted pathways. Weights represent estimated walking time in minutes. Your implementation must support a map of at least this size.

```
Sample Campus Map (weights = walking time in minutes)

  [Library] ---3--- [Science A]
      |                   |   \
      4                   2    5
      |                   |     \
    [ICT] ----6---- [ENG Block]  [Gym]
      |    \                |
      3     7               2
      |       \             |
  [Student U]  [Parkade]  [MFH]
      |
      5
      |
  [Residence]

  Nodes = Buildings   |   Edges = Pathways   |   Numbers = Minutes
```

The diagram below shows the same campus represented as a formal weighted graph — the data structure your system will need to store and traverse:

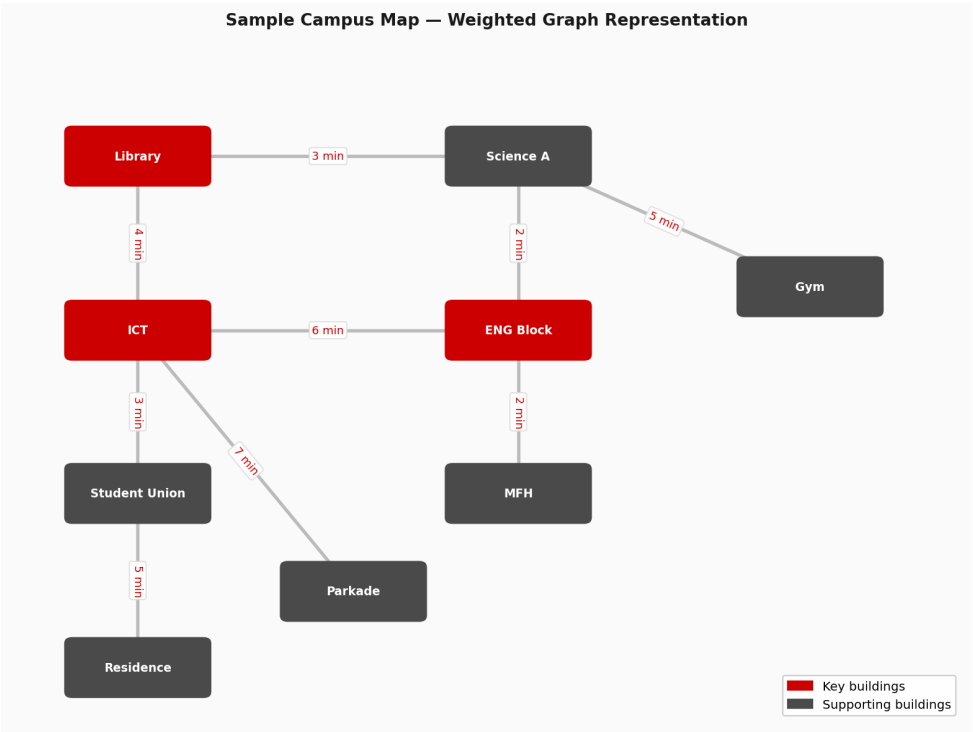

Figure 1: Campus map as a weighted undirected graph. Nodes = Building objects, edges = pathways, weights = walking time in minutes.

You may model your campus as a graph where each node is a Building object and each edge stores a path weight. The exact internal representation of the graph is a design decision you must justify in your report.

2. Required Features

Your system must implement all of the following features. For each feature, you are expected to select appropriate data organization strategies and algorithms from course material, and to justify your choices in your report.

2.1 Campus Map and Shortest Path Navigation

The campus must be represented as a network of locations (buildings, transit stops, outdoor landmarks) connected by pathways with associated travel distances or estimated travel times.

- Load the campus map from a file (e.g., a list of edges with weights).
- Given a source and destination building, compute and display the shortest path between them.
- Display the sequence of locations along the route and the total distance/time.
- Support at least 15 campus nodes and 25 edges in your test map.

2.2 Route History and Navigation Undo

Users frequently backtrack or revisit previous routes during a navigation session.

- Maintain a navigation history for each user session.
- Allow the user to undo their last navigation query and return to the previous origin.
- Support at least 10 levels of undo within a single session.

2.3 Room and Event Booking System

Campus rooms can be booked for lectures, study sessions, and events. The system must manage upcoming bookings efficiently.

- Add, remove, and retrieve room bookings by date and time.
- Query all events occurring within a given time range (e.g., “What is scheduled between 10:00 AM and 2:00 PM today?”).
- Support efficient ordered access to bookings — next upcoming event, events on a given day, etc.
- Handle at least 100 bookings in your test dataset.

2.4 Priority-Based Service Queue

Certain campus services (e.g., help desk, IT support, room maintenance requests) must be handled in order of priority rather than arrival time.

- Implement a service request queue where requests are served in order of urgency.
- Support adding new requests and always serving the highest-priority request next.

- Demonstrate the queue with at least three priority levels (e.g., Emergency, Standard, Low).

2.5 Fast Building and Resource Lookup

Users frequently need to look up information about buildings, rooms, or services by name or ID. This lookup must be fast regardless of how many records are stored.

- Store and retrieve building and room information by a unique key (name or ID).
- Support insert, delete, and lookup operations.
- Demonstrate that lookup performance is independent (or nearly independent) of the number of stored records.

2.6 Incoming Request Processing

Navigation requests and service tickets arrive continuously and must be processed in the order they are received.

- Implement a request processing pipeline that handles incoming queries in arrival order.
- Support enqueue and dequeue of navigation or service requests.
- Simulate at least 20 sequential requests being processed in order.

2.7 (Bonus) Balanced Event Index

For full marks on the bonus, implement a self-balancing index for the booking system that guarantees O(log n) insert and lookup at all times, even under adversarial insertion patterns (e.g., all bookings inserted in chronological order).

- Demonstrate the balance property is maintained after each insertion.
- Include a visualization or printout of tree structure before and after a sequence of insertions.

3. Deliverables

Each group must submit the following by the project deadline. All components are required unless otherwise noted.

3.1 Source Code

- A complete, working Python 3 codebase implementing all required features in Section 2.
- Code must be organized into logical modules (e.g., one module per major component).
- Include a README.md with clear instructions on how to run the application and reproduce the demo scenarios.
- All code must be original. Use of external libraries is permitted only for I/O and visualization; all core data structure and algorithm implementations must be your own.

3.2 Technical Report

A written report (PDF, minimum 6 pages, maximum 15 pages) structured as follows:

1. Introduction: Brief overview of the system and its purpose.
2. Design Decisions: For each required feature, describe:
   - The data structure or algorithm you chose to implement it.
   - Why this choice is appropriate for the problem (access patterns, insertion frequency, ordering requirements, etc.).
   - At least one alternative you considered and why you rejected it.
3. Complexity Analysis: For each major operation in your system, provide:
   - Time complexity (best, average, and worst case where applicable).
   - Space complexity.
   - A brief justification for each bound.
4. Challenges and Lessons Learned: Describe any significant implementation challenges and what you learned.
5. Contributions: A table listing each group member and their primary contributions to the project.

3.3 Demo Scenarios and Screenshots

Your report must include a dedicated section with screenshots of your running application, organized by scenario. Include at minimum the following use cases:

- Shortest path query: Show at least two route queries with different source/destination pairs, displaying the full path and total cost.
- Undo navigation: Show a sequence of route queries followed by one or more undo operations, demonstrating correct state restoration.
- Booking range query: Show a time-range query returning all bookings within a specified window.

- Priority queue demo: Show requests being inserted at different priority levels, then dequeued in correct priority order.
- Fast lookup demo: Show insert and lookup operations on the building/resource store, including a lookup for a key that does not exist.
- Request pipeline demo: Show at least 10 requests being enqueued and dequeued in arrival order.

Each screenshot must be accompanied by a one-to-two sentence caption explaining what is being demonstrated and which feature/component is in use.

3.4 Submission Instructions

- Your codebase must be maintained in a Git version control repository hosted on GitHub (or another approved platform). Regular commits are expected throughout the project — a repository with a single or very small number of commits will be penalized.
- Submit the URL of your public GitHub repository via the course D2L dropbox. Ensure the repository is accessible to the course instructor before the deadline.
- In addition to the GitHub link, submit a single .zip archive named ENSF338_Project_Group<N>.zip (replace <N> with your group number) containing: /src (all Python source files), /report (PDF report), and README.md at the root level.
- Submit the .zip via the course D2L dropbox before the posted deadline. Late submissions will be penalized 10% per day.
- The README.md must include the GitHub repository URL, a list of group members, and clear instructions to run the application.

4. Grading Rubric

Component	Weight	Evaluation Criteria
Working Implementation	**50%**	All required features (Section 2) are implemented, functional, and demonstrable. Code is well-organized and runs without errors.
Technical Report	**25%**	Clear design justifications, accurate complexity analysis, well-structured writing. Covers all required sections.
Demo Scenarios & Screenshots	**15%**	All six required scenarios are demonstrated with clear, captioned screenshots.
Code Quality	**10%**	Readable code, meaningful variable names, appropriate comments, modular structure.
Bonus: Balanced Index	**+5%**	Correct self-balancing implementation with demonstrated balance property.

5. Academic Integrity

All submitted work must represent the honest effort of your group. You may discuss high-level design ideas with other groups, but all code, analysis, and writing must be produced independently by your group. Submitting code copied from another group or passing off another group's work as your own is considered academic misconduct and will be handled according to the University of Calgary Student Academic Integrity Policy.

Use of Online Resources: You are permitted and encouraged to consult online references, including textbooks, documentation, tutorials, Stack Overflow, and academic papers, to understand concepts and techniques. However, all such sources must be cited in your report using standard academic referencing. Code copied or substantially adapted from an online source must also be cited inline with a comment identifying the source URL.

Use of AI Tools: You are permitted to use AI-based tools (e.g., ChatGPT, GitHub Copilot, Claude) to support your work — for example, to clarify concepts, generate test data, suggest variable names, or help debug code. However, any use of AI must be disclosed transparently. For each instance of AI use, you must include an appendix in your report titled "AI Use Disclosure" that contains:

- The name of the tool used (e.g., ChatGPT-4o, GitHub Copilot).
- The full prompt or instruction provided to the AI.
- The complete response or output generated by the AI.
- A brief explanation of how you used or modified the AI output in your project.

Undisclosed AI use — particularly for implementing core data structures or algorithms — constitutes academic misconduct. The intent of this policy is not to prohibit AI tools but to ensure you understand and can explain every component of your submission. You may be asked during a demo or office hour to explain any part of your code.

6. Tips for Success

- Start with the campus map and shortest path feature — it is the most complex and is used by other components.
- Design your interfaces (function signatures, class APIs) before writing implementation code. A clear design up front saves significant refactoring time later.
- Build incrementally and test each feature in isolation before integrating.
- For the complexity analysis, reason from first principles — do not simply look up standard results and copy them. Explain why a bound applies to your specific implementation.
- Use Python type hints and docstrings. Well-documented code is easier to debug and earns higher code quality marks.
- Commit your code to a version control repository (e.g., GitHub) regularly. This protects against data loss and makes contribution tracking straightforward.

Good luck — we look forward to seeing your systems in action!